Mini-Project 3

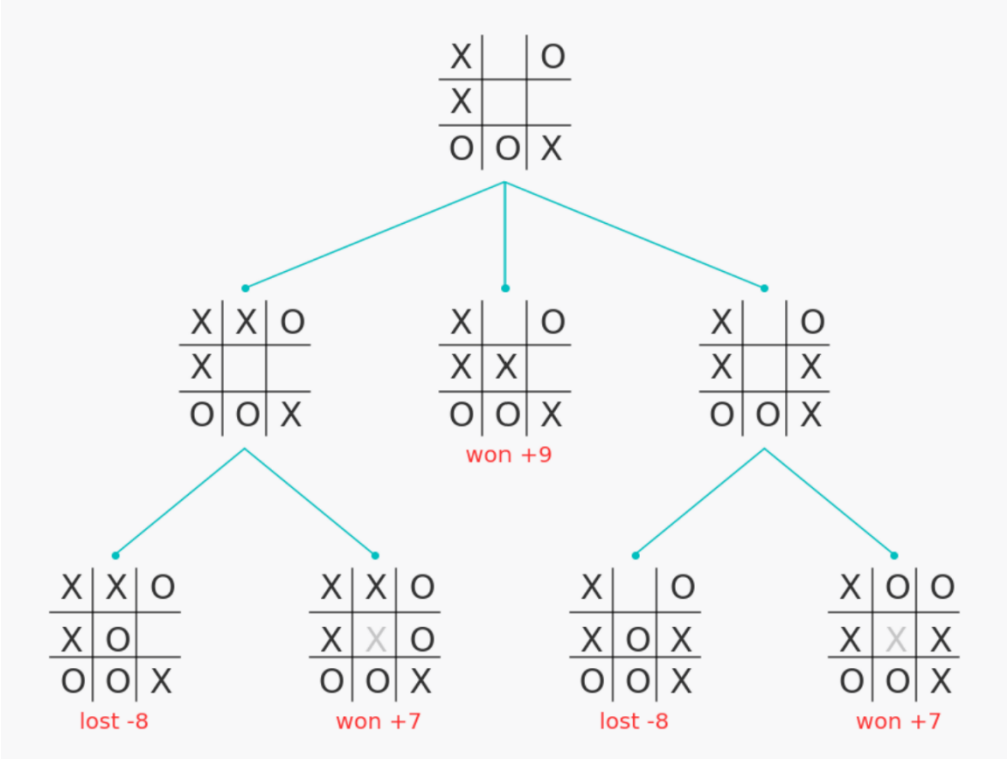
Adversarial Search for Tic-Tac-Toe

Part 1: Minimax with Alpha-Beta Pruning

Part 2: Monte Carlo Tree Search

Reference: Artificial Intelligence: A Modern Approach, 4th Edition, Sections 5.2 to 5.4

Language: Python 3



Submission deadline: April 2nd at 11:59 PM
Presentation deadline April 2nd at class time.
No presentation, no grade.

Rubric

Component	Participant	% Participation
Part 0: Game engine, correct and tested		
Part 1: Basic minimax, correct play		
Part 1: Alpha-beta pruning		
Part 1: Node counts and questions		
Part 2: MCTS, four steps correct		
Part 2: Experiments and questions		
Part 3: Tournament, timing, and analysis		
Total		

The code is provided as a sketch for your reference; it does not mean it is going to work without errors.

Introduction

In this mini-project, you will implement two AI agents that play Tic-Tac-Toe. Each agent uses a different adversarial search algorithm from Chapter 5 of the textbook.

Part 1 covers Minimax with Alpha-Beta Pruning (Sections 5.2 and 5.3). This is a deterministic algorithm that explores the complete game tree, but prunes branches that cannot influence the final decision.

Part 2 covers Monte Carlo Tree Search (Section 5.4). This is a stochastic, simulation-based algorithm that estimates the quality of each move by repeatedly playing random games to completion.

Both agents play the same game, which allows you to compare their behavior, their speed, and their results directly. Tic-Tac-Toe is small enough that you can verify correctness by hand, yet it exercises every component of each algorithm.

By completing both parts, you will understand the following:

- How minimax guarantees optimal play through exhaustive search.
- How alpha-beta pruning reduces the number of nodes explored without affecting the result.
- How MCTS estimates move quality through random simulations.
- The exploration versus exploitation tradeoff, captured by the UCB1 formula.
- The practical differences between exact and approximate methods for game playing.

Part 0: Shared Game Engine

Both assignments share the same Tic-Tac-Toe engine. You should implement this part first and test it thoroughly before proceeding to either assignment.

Task 0.1: Implement the TicTacToe Class

Create a file called game.py with the following interface. Each method includes a brief description of what it should do.

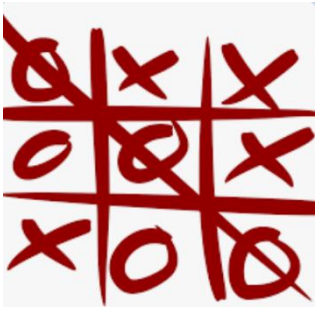
```
class TicTacToe:
    def __init__(self):
        # Initialize an empty 3x3 board.
        # Board is a list of 9 elements:
        #   0 = empty, 1 = X, -1 = O
        # X always moves first.
        self.board = [0] * 9
        self.current_player = 1

    def get_legal_moves(self):
        # Return a list of indices (0 through 8)
        # corresponding to the empty squares.
        pass

    def make_move(self, move):
        # Place the current player's mark at the
        # given index. Return a NEW TicTacToe
        # object; do not modify self.
        pass

    def is_terminal(self):
        # Return True if the game is over, either
        # because someone has won or because all
        # squares are filled (a draw).
        pass

    def utility(self):
        # Return +1 if X has won, -1 if O has won,
        # or 0 for a draw. Only valid when
        # is_terminal() returns True.
        pass

    def check_winner(self):
        # Return 1 if X has three in a row, -1 if
        # O has three in a row, or 0 otherwise.
        pass

    def display(self):
        # Print the board in a readable 3x3 format.
        # Use 'X' for 1, 'O' for -1, '.' for 0.
```

```
        pass
```

Implementation Hints

- The eight winning lines are: rows (0,1,2), (3,4,5), (6,7,8); columns (0,3,6), (1,4,7), (2,5,8); and diagonals (0,4,8), (2,4,6).
- The method make_move must return a new object so that the search algorithms can explore different branches without corrupting the current state.
- Test your engine manually before moving on. Play a few games by calling make_move in sequence, print the board, and verify that is_terminal and utility behave correctly.

0	1	2
3	4	5
6	7	8



Assignment 1: Minimax with Alpha-Beta Pruning

1.1 Background

The minimax algorithm (Section 5.2) computes the optimal move by recursively evaluating all possible future game states. The player called MAX chooses the move that maximizes the utility, while the player called MIN chooses the move that minimizes it. When applied to Tic-Tac-Toe, minimax always produces perfect play; the game always ends in a draw if both sides play optimally.

Alpha-beta pruning (Section 5.3) improves the efficiency of minimax by keeping track of two bounds as the tree is explored:

- **Alpha:** the best value that MAX can guarantee so far. It starts at negative infinity.
- **Beta:** the best value that MIN can guarantee so far. It starts at positive infinity.

Whenever alpha becomes greater than or equal to beta, the remaining children of the current node can be skipped. This pruning does not change the final result; it only avoids unnecessary work.

1.2 Task: Implement Basic Minimax

Create a file called minimax_agent.py. Begin by implementing the minimax algorithm without pruning.

```
def minimax(state):
    # Return the best move for the current player.
    if state.current_player == 1:
        best_value = float('-inf')
        best_move = None
        for move in state.get_legal_moves():
            child = state.make_move(move)
            value = min_value(child)
            if value > best_value:
                best_value = value
                best_move = move
        return best_move
    else:
        # TODO: implement the symmetric case
        # for MIN.
        pass

def max_value(state):
    if state.is_terminal():
        return state.utility()
    v = float('-inf')
    for move in state.get_legal_moves():
        child = state.make_move(move)
        v = max(v, min_value(child))
```

```
    return v

def min_value(state):
    # TODO: implement. This is symmetric to
    # max_value, but minimizes instead.
    pass
```

Verification

Once your basic minimax is working, verify the following:

1. From the empty board, minimax for X should return a corner or the center.
2. If X plays center and O plays a corner, X should respond with the opposite corner.
3. Two minimax agents playing each other should always draw.

1.3 Task: Add Alpha-Beta Pruning

Modify your minimax functions to accept alpha and beta parameters. The changes are:

4. Pass alpha and beta as additional arguments. Initialize them to negative and positive infinity, respectively.
5. In max_value, after computing the value v of a child, check whether v is greater than or equal to beta. If so, return v immediately; this is the beta cutoff.
6. In max_value, update alpha to be the maximum of alpha and v.
7. In min_value, after computing v, check whether v is less than or equal to alpha. If so, return v immediately; this is the alpha cutoff.
8. In min_value, update beta to be the minimum of beta and v.

```
def max_value_ab(state, alpha, beta):
    if state.is_terminal():
        return state.utility()
    v = float('-inf')
    for move in state.get_legal_moves():
        child = state.make_move(move)
        v = max(v, min_value_ab(child, alpha, beta))
        if v >= beta:
            return v   # Beta cutoff
        alpha = max(alpha, v)
    return v

def min_value_ab(state, alpha, beta):
    # TODO: implement with alpha cutoff.
    pass
```

1.4 Task: Count Nodes and Compare

Add a global counter that increments every time max_value or min_value is called. Run both versions (plain minimax and alpha-beta) from the empty board and report:

9. The total number of nodes evaluated by plain minimax.
10. The total number of nodes evaluated by alpha-beta.
11. The percentage of nodes pruned.

You should find that alpha-beta evaluates significantly fewer nodes while returning the same move.

1.5 Questions

Answer the following questions in a short report (one or two paragraphs each):

12. Why does minimax always produce a draw in Tic-Tac-Toe, regardless of who moves first?
13. In alpha-beta pruning, does the order in which moves are examined affect the number of nodes pruned? Why or why not?
14. What is the time complexity of minimax? What is the best-case complexity of alpha-beta? Relate your node counts to these theoretical values.
15. Would minimax be practical for a game like chess or Go? Explain your reasoning.

Assignment 2: Monte Carlo Tree Search

2.1 Background

Monte Carlo Tree Search, described in Section 5.4 of the textbook, takes a fundamentally different approach from minimax. Instead of evaluating every possible future, MCTS estimates the value of a move by simulating many random games from the current position and observing the outcomes. The value of a state is the fraction of simulations that resulted in a win.

The algorithm maintains a search tree that grows incrementally. Each iteration consists of four steps:

- **Selection.** Starting at the root, descend through the tree by choosing the child with the highest UCB1 score at each level. Continue until a leaf node is reached.
- **Expansion.** Add one new child to the selected leaf. This child represents a game state that has not been explored yet.
- **Simulation.** From the new child, play a complete game by choosing moves at random for both players until the game ends. The moves in this simulation are not recorded in the search tree.
- **Back-propagation.** Use the result of the simulation to update the statistics (wins and visit counts) of every node on the path from the new child back up to the root.

The selection step uses the UCB1 formula to balance exploitation (choosing moves with high win rates) against exploration (choosing moves that have been tried only a few times):

```
UCB1(n) = U(n)/N(n) + C * sqrt( ln(N(parent)) / N(n) )
```

Here, U(n) is the total number of wins through node n, N(n) is the total number of simulations through node n, N(parent) is the number of simulations through the parent, and C is a constant that controls the balance. A common starting value is C = 1.41 (approximately the square root of 2).

2.2 Task: Implement the MCTS Node

Create a file called mcts_agent.py. Begin by defining a node class that stores the statistics needed by the algorithm.

```
import math
import random

class MCTSNode:
    def __init__(self, state, parent=None, move=None):
        self.state = state
        self.parent = parent
        self.move = move
        self.children = []
```

```
        self.wins = 0
        self.visits = 0
        self.untried_moves = state.get_legal_moves()

    def is_fully_expanded(self):
        # Return True if every legal move from
        # this state has a corresponding child.
        pass

    def best_child(self, c=1.41):
        # Return the child with the highest UCB1
        # score. Use the formula from Section 5.4.
        pass

    def best_move(self):
        # Return the move leading to the child
        # with the most visits (not the highest
        # win rate).
        pass
```

2.3 Task: Implement the Four Steps

Implement the four core functions that correspond to the steps of the algorithm.

Selection

```
def select(node):
    # Starting from the given node, repeatedly
    # choose the best child (by UCB1) until you
    # reach a node that is not fully expanded
    # or that represents a terminal state.
    pass
```

Expansion

```
def expand(node):
    # Choose one of the untried moves, create a
    # new child node for it, and return the child.
    pass
```

Simulation

```
def simulate(state):
    # From the given state, play a random game to
    # completion. At each step, choose a uniformly
    # random legal move. Return the utility.
    pass
```

Back-propagation

```
def backpropagate(node, result):
    # Walk from the given node up to the root.
    # Increment the visit count of every node.
    # Increment the win count only for nodes
    # where the result was favorable for the
    # player who made the move.
    pass
```

2.4 Task: Implement the Main Loop

Combine the four functions into the main MCTS function, following the pseudocode from Figure 5.11 of the textbook.

```
def mcts(state, iterations=1000):
    root = MCTSNode(state)
    for _ in range(iterations):
        # 1. Select
        leaf = select(root)
        # 2. Expand (if not terminal)
        if not leaf.state.is_terminal():
            leaf = expand(leaf)
        # 3. Simulate
        result = simulate(leaf.state)
        # 4. Back-propagate
        backpropagate(leaf, result)
    return root.best_move()
```

2.5 Task: Verify Correctness

Test your MCTS agent with the following checks:

16. Run MCTS with 10,000 iterations from the empty board. It should choose the center or a corner.
17. Set up a board where X can win in one move. MCTS should find the winning move even with 100 iterations.
18. Set up a board where X must block an immediate O win. MCTS should find the blocking move.

2.6 Questions

Answer the following in a short report:

19. Run MCTS with 100, 1,000, and 10,000 iterations. How does the quality of play change? Is there a point of diminishing returns for Tic-Tac-Toe?
20. Experiment with different values of C (try 0.5, 1.0, 1.41, and 2.0). How does C affect the balance between exploration and exploitation? Report the win rates against a random player for each value.

21. Why does the algorithm return the move with the most visits rather than the highest win rate? Give a concrete example where the two criteria would select different moves.
22. MCTS does not require an evaluation function; it only needs the rules of the game. Why is this an advantage for certain games?

Part 3: Head-to-Head Comparison

In this final section, you will pit your two agents against each other and against a random player to understand their strengths and weaknesses.

3.1 Task: Build a Tournament

Write a script called tournament.py that plays multiple games between different agent combinations. Implement the following matchups, with at least 100 games per matchup:

23. Minimax (X) versus Random (O).
24. MCTS with 1,000 iterations (X) versus Random (O).
25. Minimax (X) versus MCTS with 1,000 iterations (O).
26. MCTS with 1,000 iterations (X) versus Minimax (O).

For each matchup, record the number of wins for X, wins for O, and draws. Present your results in a table.

3.2 Task: Measure Execution Time

For each agent, measure the average time per move using Python's time module. Report the average over all moves in a game.

Repeat the timing measurement for MCTS with different iteration counts (100, 500, 1000, 5000, 10000) and plot a graph of iterations versus average time per move.

3.3 Analysis Questions

Write a brief comparative analysis addressing the following:

27. Which agent wins more games against the random player? Are the results what you expected?
28. When minimax plays against MCTS, what happens? Does MCTS ever lose? If so, how many iterations does it need to avoid losing?
29. Compare the time per move for both agents. At what iteration count does MCTS become slower than minimax for Tic-Tac-Toe?
30. Consider a game with a much larger branching factor, such as Connect Four (branching factor of approximately 7, depth up to 42). Which algorithm would you choose for that game, and why?
31. Under what circumstances would you prefer MCTS over minimax? Under what circumstances would minimax be the better choice?

Deliverables

Submit the following files:

File	Description
game.py	The Tic-Tac-Toe game engine (Part 0).
minimax_agent.py	Minimax and alpha-beta pruning (Assignment 1).
mcts_agent.py	Monte Carlo Tree Search (Assignment 2).
tournament.py	Head-to-head comparison script (Part 3).
report.pdf	Written answers to all questions, tables, and graphs.

Grading Rubric

Component	Points	Percentage
Part 0: Game engine, correct and tested	15	15%
Part 1: Basic minimax, correct play	15	15%
Part 1: Alpha-beta pruning	15	15%
Part 1: Node counts and questions	10	10%
Part 2: MCTS, four steps correct	20	20%
Part 2: Experiments and questions	10	10%
Part 3: Tournament, timing, and analysis	15	15%
Total	**100**	**100%**

References

Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach, 4th Edition. Pearson. Sections 5.2, 5.3, and 5.4.

Browne, C. et al. (2012). A Survey of Monte Carlo Tree Search Methods. IEEE Transactions on Computational Intelligence and AI in Games, 4(1), pp. 1-43.